Universidad de Buenos Aires

Facultad de Ciencias Exactas y Naturales

Trabajo Práctico 3

Introducción al Modelado Continuo

Resolución numérica de una ecuación de Poisson unidimensional



Buenos Aires, Argentina

Introducción al modelado continuo
Trabajo práctico 3 - 1er. cuatrimestre 2026

Grupo: 5
Nombre del grupo: RAM.

1	2	Nota

Apellido: Alarcón
Nombre: Melina — Carrera: Lic. en Cs. de Datos

Apellido: Belbrun
Nombre: Axel — Carrera: Lic. en Cs. de Datos

Apellido: Dominguez
Nombre: Rocio Julieta — Carrera: Lic. en Cs. de Datos

Resolución numérica de una ecuación de Poisson unidimensional

Se desea resolver numéricamente la ecuación de Poisson en una dimensión con condiciones de Dirichlet en $x = 0$ y de Neumann en $x = 1$.

$$\begin{cases} u_{xx}(x) = f(x) \quad \text{para } x \in (0,1) \\ u(0) = \alpha \\ u_x(1) = \beta \end{cases} \tag{1}$$

1. Plantear el problema como un sistema lineal $Au = b$, para $f(x) = \sin(128\pi x)$, $\alpha = 2$, $\beta = 1$ en una malla uniforme $\{x_j = hj\}$ con $j = 0, 1, 2, \ldots, 2^n + 1$ y $h = \frac{1}{2^n+1}$.

   Construir la matriz A de diferenciación utilizando el esquema de diferencias centradas para la derivada segunda y diferencias hacia atrás (*backward*) de orden 2 para la condición de Neumann (es la aproximación del ejercicio 2(a)-2(b) para 3 nodos). Mostrar las formulas de las aproximaciones usadas para las derivadas. Mostrar algunas matrices A y b con n chico (que entre en papel)

2. Resolver y calcular el error en $\|\cdot\|_\infty$ respecto a la solución exacta para $k = 3, 4, \ldots, 14$ y graficar el logaritmo del error en funcion del logaritmo de h para estimar el orden del metodo. Las soluciones mostrarlas como graficos para algunos n representativos. Para el orden del error estimar la pendiente del grafico log-log

3. Repetir el primer ítem y resolverlo para $f(x) = e^{-x^2}$, $\alpha = 2$, $\beta = 1$ utilizando matrices esparsas. ¿Cuál es el mayor k que puede utilizar? ¿Por qué? Mostrar un grafico de la solucion para el n mas alto.

Entregar desarrollo en papel, archivos de gráficos y programas.

Resolución numérica de una ecuación de Poisson unidimensional

Introducción

Una ecuación en derivadas parciales, *EDP*, es aquella en la que la incógnita es una función de al menos dos variables independientes, y que en esta ecuación aparecen involucradas tanto la función como sus derivadas parciales respecto de sus variables.

En el presente trabajo se nos presentó un sistema compuesto por una ecuación diferencial y dos condiciones de contorno. Resolver este sistema implica dar a conocer la función u que satisface la ecuación dada, $u_{xx}(x) = f(x)$, que corresponde a una EDP de Poisson puesto que f no es la función cero (en tal caso sería de Laplace). Resolver también implica que nuestra solución cumple con las condiciones de contorno dadas: una de Dirichlet que nos indica el valor de u en la frontera de su dominio, y otra de Neumann que nos aporta información sobre la pendiente o derivada de nuestra función u también en la frontera.

Para dar respuesta al trabajo usamos las herramientas vistas en clase, y trabajamos con métodos de diferencias finitas que resuelven EDPs a partir la definición de una grilla de finitos nodos o puntos, que usamos para aproximar las derivadas de nuestra función y con ello construir un sistema de ecuaciones lineales en la que la incógnita resultaba ser la función buscada.

Ejercicio 1

Consideramos la ecuación de Poisson en el intervalo $(0, 1)$ con las condiciones de contorno de Dirichlet en $x = 0$ y de Neumann en $x = 1$:

$$\begin{cases} u_{xx}(x) = \sin(128\pi x) & \text{para } x \in (0,1) \\ u(0) = 2 & \text{(Dirichlet)} \\ u_x(1) = 1 & \text{(Neumann)} \end{cases} \tag{2}$$

Discretización del dominio

Consideramos una malla uniforme del intervalo $[0, 1]$ formada por $2^k + 2$ nodos,

$$x_j = jh, \qquad j = 0, 1, \ldots, 2^k + 1$$

donde

$$h = \frac{1}{2^k + 1}.$$

Como el número de intervalos es

$$N = 2^k + 1$$

también puede escribirse

$$x_j = jh, \qquad j = 0, \ldots, N,$$

con

$$h = \frac{1}{N}.$$

De esta manera obtenemos la grilla x_0=0 * h, x_1=1 * h, x_2=2 * h, ... , x_{2^k+1}=2^{k+1} * h.

Sistema de condiciones

Como nuestro objetivo es hallar u mediante aproximaciones, nos basta con buscar un sistema de condiciones que involucren las coordenadas de u.

Los valores aproximados de la solución en los nodos se denotarán por $u_j \approx u(x_j)$. Y como la condición de Dirichlet fija el valor $u_0 = 2$, las incógnitas del sistema resultan

$$\mathbf{u} = [u_1, u_2, \ldots, u_N]^T \in \mathbb{R}^N.$$

Distinción entre nodos

Dado que la ecuación de Poisson $u_{xx}(x) = \sin(128\pi x)$ estaba dada para el intervalo abierto (0,1), nos pareció importante diferenciar el nodo correspondiente a j = N, puesto que $x_N = 1$ donde ya no vale la ecuación diferencial, no obstante sí tenemos información de u_x en dicho valor gracias a la condición de Neumann. De aquí pudimos separar las condiciones que aplicaríamos a los distintos nodos.

Discretización de los nodos internos

Para los nodos internos ($j = 1, 2, \ldots, N-1$), utilizamos el esquema de diferencias centradas de orden 2 para la derivada segunda (ejercicio 4, guía de Laboratorio):

$$u_j'' \approx \frac{u_{j-1} - 2u_j + u_{j+1}}{h^2} = \sin(128\pi x) \tag{3}$$

- **Para el primer nodo interno ($j = 1$):** Sustituimos el valor de la condición de contorno izquierda ($u_0 = \alpha = 2$) y trasladamos el término conocido al miembro derecho:

$$\frac{-2u_1 + u_2}{h^2} = \sin(128\pi x) - \frac{2}{h^2} \tag{4}$$

- **Para los nodos intermedios ($j = 2, \ldots, N-1$):** La ecuación se mantiene de la forma estándar:

$$\frac{u_{j-1} - 2u_j + u_{j+1}}{h^2} = \sin(128\pi x) \tag{5}$$

Discretización de la condición de Neumann

Para incluir la condición de Neumann tuvimos en cuenta lo trabajado en clase: a partir de 3 nodos x_{i-2}, x_{i-1}, x_i construimos un polinomio de Lagrange tal que pase por (x_{i-2}, u_{i-2}), (x_{i-1}, u_{i-1}) y por (x_i, u_i).

Polinomio de Lagrange para 3 nodos

Construimos el Polinomio de Lagrange a partir de otros tres polinomios:

- L_0 es el polinomio que vale 1 en x_i y se anula en los otros puntos, es decir $L_0(x) = \frac{(x-x_{i-1})(x-x_{i-2})}{(x_i-x_{i-1})(x_i-x_{i-2})}$.
- L_1 es el polinomio que vale 1 en x_{i-1} y se anula en los otros puntos, es decir $L_1(x) = \frac{(x-x_i)(x-x_{i-2})}{(x_{i-1}-x_i)(x_{i-1}-x_{i-2})}$.
- L_2 es el polinomio que vale 1 en x_{i-2} y se anula en los otros puntos, es decir $L_2(x) = \frac{(x-x_i)(x-x_{i-1})}{(x_{i-2}-x_i)(x_{i-2}-x_{i-1})}$.

Con ellos construimos nuestro polinomio de Lagrange:

$$L(x) = u_i * L_0(x) + u_{i-1} * L_1(x) + u_{i-2} * L_2(x),$$

que por construcción pasa por (x_{i-2}, u_{i-2}), (x_{i-1}, u_{i-1}) y por (x_i, u_i).

Esquema de diferencias hacia atrás para la derivada de *u*

Derivamos nuestro polinomio de Lagrange y evaluamos en x_i para obtener

$$L'(x_i) = \frac{3 * u_i - 4 * u_{i-1} + u_{i-2}}{2h},$$

la aproximación de *u'(x)* centrada en x_i a partir de sus dos nodos anteriores x_{i-2} y x_{i-1}.

Condición de Neumann

Como es en el último nodo ($x_N = 1$) donde disponemos de la condición $u_x(1) = \beta$, usamos el esquema de diferencias hacia atrás (*backward*) de orden 2 empleando 3 nodos: u_{N-2}, u_{N-1}, y u_N, obteniendo:

$$u'(x_N) = \frac{3u_N - 4u_{N-1} + u_{N-2}}{2h} \tag{6}$$

y como $u'(x_N) = u'(1)$, entonces $u'(x_N) = \beta = 1$, y tenemos:

$$\frac{3u_N - 4u_{N-1} + u_{N-2}}{2h} = 1. \tag{7}$$

Reordenando los términos según el índice de las variables, la última condición de nuestro sistema resulta

$$\frac{1}{2h}u_{N-2} - \frac{4}{2h}u_{N-1} + \frac{3}{2h}u_N = 1. \tag{8}$$

Construcción del sistema lineal Au = b

Habiendo hallado a grandes rasgos un sistema de condiciones que involucran las coordenadas de u:

- u_0 no es incógnita.
- Podemos obtener $u''(x_i)$ para los nodos j = 1, ..., N con combinaciones lineales de las coordenadas de u.
- Podemos obtener $u'(x_N)$ como combinación lineal de sus nodos anteriores.

Planteamos un sistema de ecuaciones lineales representable mediante una matriz **A** que guardará las operaciones entre coordenadas antes mencionadas y un vector **b** de resultados. Así,

$$\mathbf{A} * \boldsymbol{u} = \mathbf{b}$$

.

Nuestra incógnita es u y queremos resolver el siguiente sistema para hallarla:

$$\begin{bmatrix} -\frac{2}{h^2} & \frac{1}{h^2} & 0 & \dots & 0 & 0 \\ \frac{1}{h^2} & -\frac{2}{h^2} & \frac{1}{h^2} & \dots & 0 & 0 \\ 0 & \ddots & \ddots & \ddots & \vdots & \vdots \\ 0 & \dots & \frac{1}{h^2} & -\frac{2}{h^2} & \frac{1}{h^2} & 0 \\ 0 & \dots & 0 & \frac{1}{h^2} & -\frac{2}{h^2} & \frac{1}{h^2} \\ 0 & \dots & 0 & \frac{1}{2h} & -\frac{4}{2h} & \frac{3}{2h} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_{N-2} \\ u_{N-1} \\ u_N \end{bmatrix} = \begin{bmatrix} \sin(128\pi x) - \frac{2}{h^2} \\ \sin(128\pi x) \\ \vdots \\ \sin(128\pi x) \\ \sin(128\pi x) \\ 1 \end{bmatrix} \tag{9}$$

Implementación

Mediante las bibliotecas `Numpy` y `Scipy` construimos nuestra matriz **A** de dos maneras:

- Matriz densa: hecha con las funciones para vectores de `Numpy`.
- Matriz esparza: hecha con la función `diags_array` de `scipy.sparse`.

El vector **b** fue hecho con *Numpy* dado que de resolver el sistema Au=b con un solve y A esparzo, no se necesita que b también lo sea.

Ejemplo: k = 2

Para k = 2 tenemos los índices j = 0, 1, 2, ..., $2^2 + 1$, es decir, j = 0, 1, 2, 3, 4, 5 y h = $\frac{1}{5} = 0.2$.
De esta manera, tenemos la malla x = {0, 0.2, 0.4, 0.6, 0.8, 1} que discretiza nuestro dominio [0,1].
Siguiendo las condiciones dadas anteriormente, buscamos conocer u_1, u_2, u_3, u_4 y u_5.
Para las filas de condiciones correspondientes a los índices 1 a 4 teníamos una distinción:

- El nodo 1 usa la condición $\frac{-2u_1+u_2}{h^2} = \sin(128\pi x_1) - \frac{2}{h^2}$, es decir:

$$\frac{-2u_1 + u_2}{0{,}2^2} = \sin(128\pi 0{,}2) - \frac{2}{0{,}2^2} \tag{10}$$

- Los nodos j = 2, 3, 4 usan la condición $\frac{u_{j-1}-2u_j+u_{j+1}}{h^2} = \sin(128\pi x_j)$, es decir:

$$\frac{u_{j-1} - 2u_j + u_{j+1}}{h^2} = \frac{u_{j-1} - 2u_j + u_{j+1}}{0{,}2^2} = \sin(128\pi x_j) \tag{11}$$

Por último, la última fila de nuestras condiciones estaba dada por la condición de Neumann, entonces:

$$\frac{3u_5 - 4u_4 + u_3}{2h} = \frac{3u_5 - 4u_4 + u_3}{2 * 0{,}2} = 1. \tag{12}$$

Nuestro sistema **A*u = b** para k=2 resulta:

$$\begin{bmatrix} -\frac{2}{0{,}2^2} & \frac{1}{0{,}2^2} & 0 & 0 & 0 \\ \frac{1}{0{,}2^2} & -\frac{2}{0{,}2^2} & \frac{1}{0{,}2^2} & 0 & 0 \\ 0 & \frac{1}{0{,}2^2} & -\frac{2}{0{,}2^2} & \frac{1}{0{,}2^2} & 0 \\ 0 & 0 & \frac{1}{0{,}2^2} & -\frac{2}{0{,}2^2} & \frac{1}{0{,}2^2} \\ 0 & 0 & \frac{1}{2*0{,}2} & -\frac{4}{2*0{,}2} & \frac{3}{2*0{,}2} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \end{bmatrix} = \begin{bmatrix} \sin(128\pi 0{,}2) - \frac{2}{0{,}2^2} \\ \sin(128\pi 0{,}4) \\ \sin(128\pi 0{,}6) \\ \sin(128\pi 0{,}8) \\ 1 \end{bmatrix} \tag{13}$$

Ejercicio 2

Cálculo de la solución exacta

Para evaluar la precisión del método numérico y estimar su orden de convergencia, primero hallamos la solución analítica exacta del problema de contorno mediante integración directa de la ecuación diferencial:

$$u_{xx}(x) = \sin(128\pi x) \tag{14}$$

1. **Primera integración:** Integramos respecto a x para obtener la expresión de la primera derivada:

$$u_x(x) = \int \sin(128\pi x)\, dx = -\frac{\cos(128\pi x)}{128\pi} + C_1 \tag{15}$$

2. **Aplicación de la condición de Neumann ($x = 1$):** Sabiendo que $u_x(1) = 1$, determinamos el valor de la constante C_1:

$$u_x(1) = -\frac{\cos(128\pi 1)}{128\pi} + C_1 = 1 \implies C_1 = 1 + \frac{1}{128\pi} \tag{16}$$

   De este modo, la derivada primera queda definida como $u_x(x) = -\frac{\cos(128\pi x)}{128\pi} + 1 + \frac{1}{128\pi}$.

3. **Segunda integración:** Integramos la expresión anterior para hallar la función $u(x)$:

$$u(x) = \int -\frac{\cos(128\pi x)}{128\pi} + 1 + \frac{1}{128\pi}\, dx = -\frac{sin(128\pi x)}{(128\pi)^2} + x(1 + \frac{1}{128\pi}) + C_2 \tag{17}$$

4. **Aplicación de la condición de Dirichlet ($x = 0$):** Utilizando que $u(0) = 2$, despejamos la constante C_2:

$$u(0) = -\frac{sin(128\pi 0)}{(128\pi)} + 0(1 + \frac{1}{128\pi}) + C_2 = 2 \implies C_2 = 2 \tag{18}$$

Por lo tanto, la solución exacta del problema es:

$$u(x) = \frac{-\sin(128\pi x)}{(128\pi)^2} + (1 + \frac{1}{128\pi})x + 2 \tag{19}$$

Implementación numérica

Para hallar u usamos las matrices $\mathbf{A}$, $\mathbf{b}$ del sistema lineal obtenido anteriormente.

Lo implementamos en `Python` utilizando dos enfoques: matrices densas y matrices esparzas. En ambos casos se resolvió el sistema correspondiente y se comparó la solución numérica con la solución exacta.

Resolución del sistema $\mathbf{Au} = \mathbf{b}$

Decidimos resolver el sistema $\mathbf{Au} = \mathbf{b}$ de dos maneras:

- Caso denso: usamos `solve` de la biblioteca `scipy.linalg`.
- Caso esparzo: usamos `spsolve` de la biblioteca `scipy.sparce.linalg`.

Más allá de pequeños errores numéricos propios de los métodos de aproximación utilizados y quizás diferencias entre las funciones o la forma de almacenar los datos, las soluciones aproximadas para u de ambos métodos resultaron idénticas tanto para nuestra percepción como para los errores que calculamos posteriormente respecto de la solución exacta.

Mostramos la solución u hallada por los métodos para k=7 y los comparamos con la solución exacta.

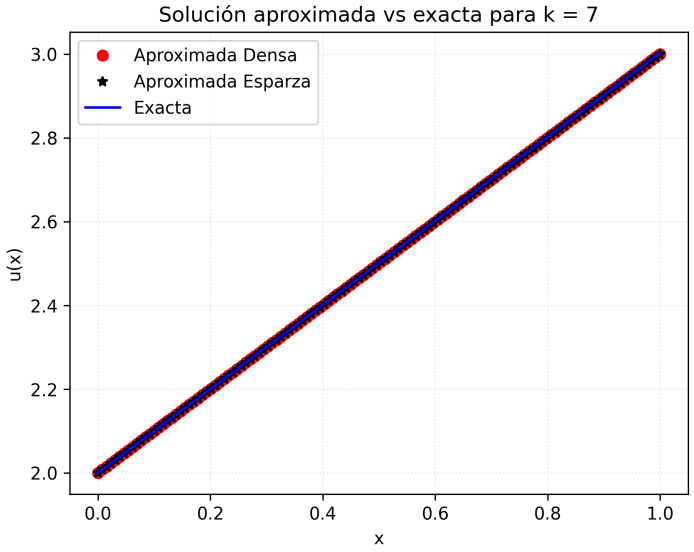


Figura 1: Comparativa entre las soluciones encontradas por los métodos y la solución exacta para k = 7.

Notamos que con k = 7 los métodos aproximan muy bien, se pegan a la solución exacta y entre ellos no notamos diferencia.

Medición de tiempos

Adicionalmente, hicimos uso de la biblioteca `time` para medir y comparar los tiempos de ejecución de los *solve* para el caso denso y esparzo obteniendo:

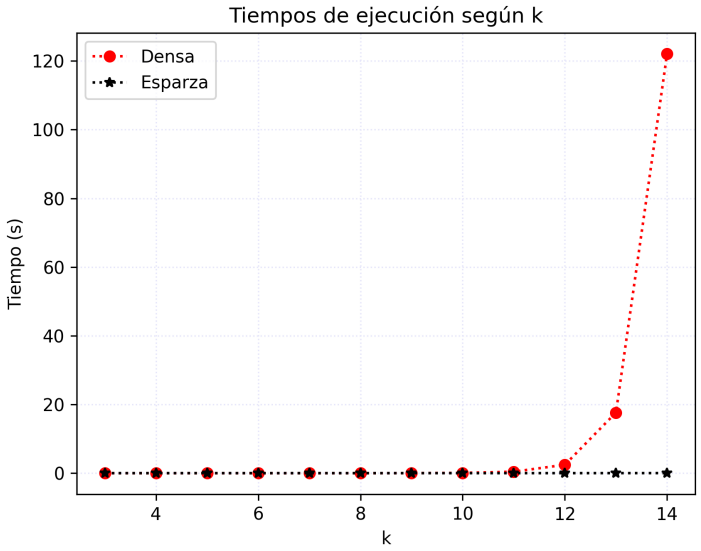


Figura 2: Comparativa entre los tiempos de ejecución de un sistema lineal para matrices densas y esparzas con k entre 3 y 14.

Comprobamos que a partir de un k $= 11$ los tiempos de ejecución comienzan a separarse, siendo el método esparzo mucho más eficiente.

Al realizar ejecuciones sobre este ejercicio notamos cómo el uso de matrices densas nos demoraba por los tamaños de las matrices involucradas al aumentar el k.

Análisis del error y orden de convergencia

Para estudiar experimentalmente el orden de convergencia del esquema implementado, se evaluó el error global en norma infinito $\|\cdot\|_\infty$ barriendo los niveles de discretización desde $k = 3$ hasta $k = 14$.

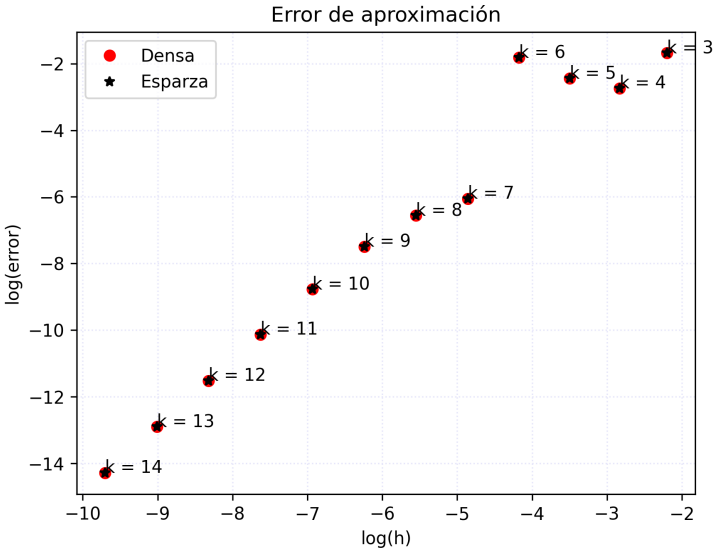


Figura 3: Evolución del error global de aproximación log(error) en función de log(h) para matrices densas y esparsas en los niveles $k = 3, \ldots, 14$.

A partir de los resultados obtenidos en las simulaciones numéricas, se observa que tanto el enfoque de almacenamiento denso como el esparzo arrojan aproximaciones idénticas, notando que a medida que k aumenta el error se vuelve más pequeño.

Adicionalmente, vemos que a partir de k = 7 se nota una mejora en la aproximación de la solución, lo que tiene sentido respecto a lo obtenido en la Figura 1 que comparaba las soluciones para k = 7.

Aproximación y regresión lineal

Como anteriormente notamos que ambos métodos daban resultados que diríamos iguales, decidimos estudiar el orden del método denso en representación de ambos.

Para estimar con precisión el orden de convergencia experimental de dicho método sin la influencia de distorsiones por efectos de borde en grillas demasiado gruesas, se realizó un ajuste lineal por cuadrados mínimos utilizando la función `polyfit` de la biblioteca `Numpy` sobre los datos de errores del método denso correspondientes a $k \geq 7$ (es decir, a partir del índice 4 del vector de pasos espaciales).

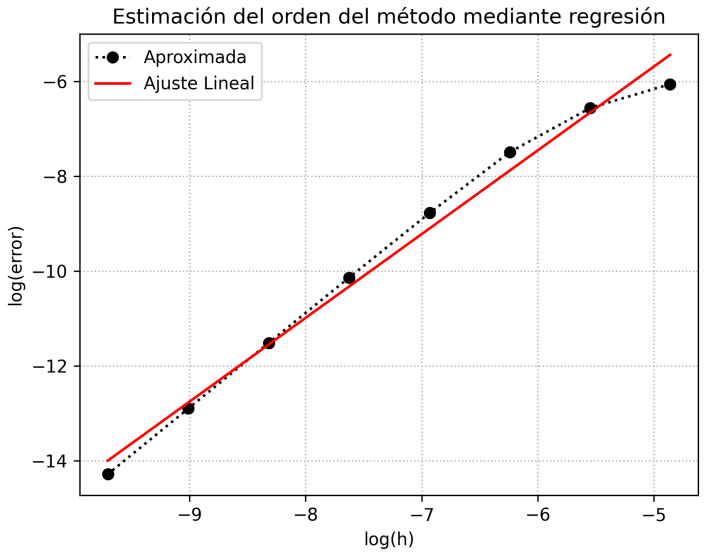


Figura 4: Estimación del orden del método mediante regresión lineal sobre el subconjunto de grillas finas.

La pendiente del ajuste lineal resultante es **1.7677**. Este valor representa el orden de convergencia experimental del método denso y esparzo, y ratifica la consistencia del esquema numérico propuesto frente al orden teórico esperado $\mathcal{O}(h^2)$.

Ejercicio 3

En este ejercicio, planteamos nuevamente el sistema con la $f(x)$ dada, quedándonos el siguiente:

$$\begin{cases} u_{xx}(x) = e^{-x^2} & \text{para } x \in (0,1) \\ u(0) = 2 & \text{(Dirichlet)} \\ u_x(1) = 1 & \text{(Neumann)} \end{cases} \tag{20}$$

Para este ítem, el sistema lineal $A\mathbf{u} = \mathbf{b}$ mantiene exactamente la misma estructura de la matriz de diferenciación A construida en el Ejercicio 1, dado que las condiciones de contorno ($\alpha = 2, \beta = 1$) y el operador de derivación no han cambiado. La única modificación radica en el vector de carga $\mathbf{b}$, debido al cambio en la función $f(x) = e^{-x^2}$.

El sistema resultante para un tamaño $N = 2^k + 1$ toma la forma:

$$\begin{bmatrix} -\frac{2}{h^2} & \frac{1}{h^2} & 0 & \dots & 0 & 0 \\ \frac{1}{h^2} & -\frac{2}{h^2} & \frac{1}{h^2} & \dots & 0 & 0 \\ 0 & \ddots & \ddots & \ddots & \vdots & \vdots \\ 0 & \dots & \frac{1}{h^2} & -\frac{2}{h^2} & \frac{1}{h^2} & 0 \\ 0 & \dots & 0 & \frac{1}{h^2} & -\frac{2}{h^2} & \frac{1}{h^2} \\ 0 & \dots & 0 & \frac{1}{2h} & -\frac{4}{2h} & \frac{3}{2h} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_{N-2} \\ u_{N-1} \\ u_N \end{bmatrix} = \begin{bmatrix} e^{-x_1^2} - \frac{2}{h^2} \\ e^{-x_2^2} \\ \vdots \\ e^{-x_{N-1}^2} \\ e^{-x_N^2} \\ 1 \end{bmatrix} \tag{21}$$

Dado que no se dispone de una primitiva de e^{-x^2}, la resolución se realiza de manera puramente numérica a partir de la discretización directa.

Análisis del límite para el valor de k

Al implementar este problema utilizando matrices esparzas mediante la estructura `diags_array` en formato comprimido por columnas, se redujo drásticamente el uso de memoria RAM de una complejidad espacial cuadrática $\mathcal{O}(N^2)$ a una estrictamente lineal $\mathcal{O}(N)$, dado que el almacenamiento conserva únicamente los elementos no nulos de la estructura tridiagonal.

Para determinar experimentalmente el límite absoluto de la arquitectura de hardware de la máquina local, se diseñó un algoritmo automático de prueba de estrés incrementando gradualmente el valor de k. En la Cuadro 1 se detallan los tiempos de ejecución y la dimensión del sistema para cada nivel evaluado.

Análisis del límite para el valor de k

Al implementar este problema utilizando matrices esparsas mediante la función `diags_array` de la biblioteca `scipy.sparse` en formato comprimido por columnas (`CSC`), se redujo drásticamente el uso de memoria RAM de una complejidad espacial cuadrática $\mathcal{O}(N^2)$ a una estrictamente lineal $\mathcal{O}(N)$, dado que el almacenamiento conserva únicamente los elementos no nulos de la estructura tridiagonal.

Para determinar experimentalmente el límite absoluto de la arquitectura de hardware de la máquina local, se diseñó un algoritmo automático de prueba de estrés incrementando gradualmente el valor de k. En la Cuadro 1 se detallan los tiempos de ejecución y la dimensión del sistema para cada nivel evaluado en la última corrida reproducible.

Cuadro 1: Resultados de la prueba de estrés utilizando matrices esparsas.

k	N	**Tiempo de ejecución (s)**	**Estado**
14	16.385	0.0000	Éxito
15	32.769	0.0156	Éxito
16	65.537	0.0424	Éxito
17	131.073	0.0746	Éxito
18	262.145	0.1497	Éxito
19	524.289	0.2655	Éxito
20	1.048.577	0.5982	Éxito
21	2.097.153	1.1061	Éxito
22	4.194.305	3.0491	Éxito
23	8.388.609	—	Error de Memoria

Como se puede apreciar, el mayor nivel resuelto con éxito de forma eficiente en esta sesión fue $k = 22$, procesando un sistema 4.194.305 de ecuaciones simultáneas en poco más de 3 segundos.

A continuación, se presenta el perfil de la solución numérica calculada para esta máxima resolución lograda:

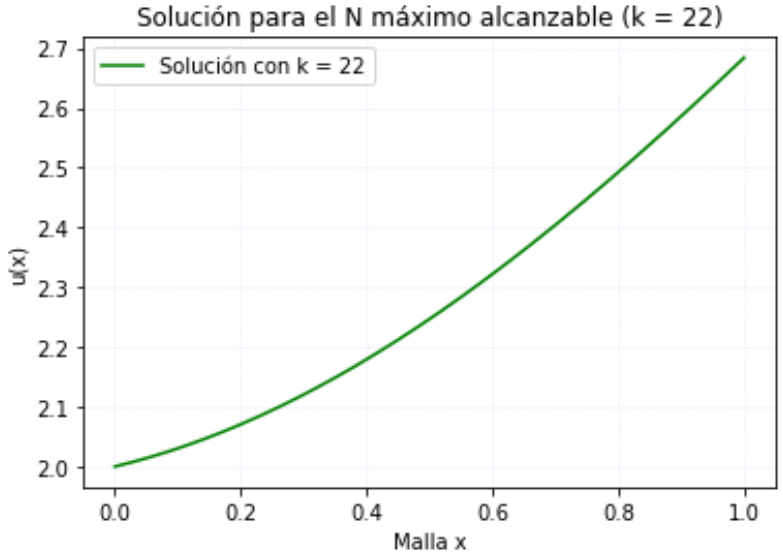


Figura 5: Solución aproximada para el máximo N alcanzable ($k = 22$) obtenida mediante el solver esparzo.

Análisis del resultado numérico

Si bien no tenemos manera de hallar la solución exacta analítica para este problema, notamos que al estar frente a una *EDP* unidimensional, u_{xx} es Δu.

En clases teóricas estudiamos que el laplaciano de una función, Δu, nos indica su curvatura espacial o concavidad. En nuestro problema tenemos que $\Delta u = e^{-x^2}$ que es siempre positiva, por lo cual consideramos que la u resultante debía notarse cóncava hacia arriba lo cual coincide con el gráfico de la solución hallada numéricamente.

Explicación del error de memoria

Al intentar evaluar el nivel $k = 23$ (que implica duplicar la grilla a más de 8 millones de incógnitas), el entorno interrumpió la ejecución arrojando la siguiente excepción de la biblioteca subyacente:

```
SUPERLU_MALLOC fails for buf in intCalloc()
```

Este error específico no proviene directamente de las estructuras nativas de Python, sino de la subrutina en C de `SuperLU`, que es el solver directo por defecto que utiliza SciPy (`spsolve`) para matrices esparsas no simétricas. A pesar de que el almacenamiento de la matriz A es $\mathcal{O}(N)$, el método de resolución directa realiza una factorización LU (eliminación gaussiana esparza). Durante esta etapa se produce un fenómeno conocido como *fill-in* (aparición de elementos no nulos en posiciones que originalmente eran cero).

Para un sistema de la magnitud de $k = 23$, la cantidad de memoria dinámica requerida para direccionar los punteros del espacio de trabajo indexado por `intCalloc()` superó la capacidad física de la memoria RAM asignable de forma contigua por el sistema operativo, provocando el fallo del buffer y el consecuente colapso de la simulación.

Conclusión

Los métodos numéricos para aproximar soluciones a EDPs pueden ser eficientes con las bibliotecas adecuadas. Vimos la ventaja que se tiene al trabajar con matrices esparzas ante grandes cantidades de datos, y cómo aún siendo una solución aproximada, resulta fiel a la solución exacta (en el caso que podía calcularse).

Incluso con matrices esparzas resulta importante tener en cuenta las limitaciones de disco duro y ram propias de las máquinas en que corremos nuestro programa.

Realizar este trabajo nos permitió abordar lo trabajado en las clases, programar con bibliotecas optimizadas para estos problemas e investigar sobre las ecuaciones en derivadas parciales.

Fuentes y Bibliografía

- Introducción al Modelado Continuo (Notas de clase)- Julián Fernández Bonder https://drive.google.com/file/d/1WY9QK7aCJwJMjMGBhxn5u8Z24XBUaGLf/view
- Szretter, M. *Apunte de Regresión Lineal*. Universidad de Buenos Aires (UBA). Disponible en: https://mate.dm.uba.ar/~meszre/apunte_regresion_lineal_szretter.pdf
- Cátedra de Análisis Numérico (2do Cuatrimestre 2020). *Apunte de la Materia*. Departamento de Matemática, FCEN, UBA. Disponible en: https://cms.dm.uba.ar/academico/materias/2docuat2020/Analisis_Numerico/apunte.pdf
- Modelo de IA Gemini. *Se consultó para la lógica de interrupción por timeout y depuración del error de asignación de memoria de SuperLU en SciPy*
- Ecuación en derivadas parciales (Artículo de Wikipedia). https://es.wikipedia.org/wiki/Ecuacion_en_derivadas_parciales